

Money Guide - Spring 2025

Joseph Lilly Jr. | Project 1

Summary

This Python program asks for monthly income, expenses, and debt payments, then estimates how much money is available to save or invest. It compares regular savings, a high-yield savings account, an S&P 500 estimate, and a real estate investment estimate over one-year and three-year timelines.

Source Code

```
'''
Project 1 - Money Guide - Spring 2025
Author: Joseph Lilly Jr.  Joseph24

This program has 5 functions and helps with money decisions. The basic concept is that if you
are spending more than you earn, you are making poor
financial decisions. If you are doing life correctly and getting more money back then you
spend, this program calculates how much money you are
available to invest each month, how much money you will have at the end of the year, your high-
yield savings on that money, your S&P 500 investment,
and your Real Estate Investment.

I have neither given or received unauthorized assistance on this assignment.
Signed:  Joseph Lilly Jr.
'''

import math

'''
calculate_spare_cash function gets the user's monthly income, monthly expenses, and monthly
debt to calculate potential savings called spare_cash.
Parameters:
    None
Returns:
    potential savings
'''

def calculate_spare_cash():
    monthly_income = input('Enter your monthly income or allowance ($): ')
    monthly_expense = input('Enter your total monthly expenses ($): ')
    monthly_debt = input('Enter your monthly debt payment ($), or 0 if none: ')
    spare_cash = float(monthly_income) - float(monthly_expense) - float(monthly_debt) #
    potential_savings = (amount of money owned) - (spending total)
    return round(spare_cash, 2)

'''

calculate_savings function finds the potential savings using the potential savings found from
the user each month stored in the variable spare_cash.
Parameters:
    savings, annual_interest, months
Return:
    final savings for bank account
'''

def calculate_savings(savings, annual_interest, months):
    final_savings = 0 # initializing final_savings
    compound_rate = (annual_interest / 12)
    for month in range(months): # compounds the interest for each month. (range months)
        final_savings += savings
        final_savings = final_savings * compound_rate + final_savings

    return round(final_savings, 2)
```

```

'''
calculate_sp500_investment function finds the investment the user would have if all spare_cash
each month went into an sp500.
Parameters:
    savings, annual_return, dividend_yield, months
Return:
    sp500 investment value
'''

def calculate_sp500_investment(annual_return, savings, dividend_yield, months):
    monthly_growth_rate = ((annual_return + dividend_yield) / 12) # calculates how much
    money grows each month
    investment_value = 0 # initializing
    for month in range(months): # compounding the same way as before but with the monthly
    growth rate.
        investment_value += savings
        investment_value = investment_value * monthly_growth_rate + investment_value
    return round(investment_value, 2)

'''

calculate_real_estate_investment function finds the investment value the user would have if
they went into real estate. It facors in home appreciation,
rental income, downpayment on morgage and more.
Parameters:
    savings, home price, rental income, annual appreciation, mortgage rate, years of
    loan, and the length of term
Return:
    real estate investment value
'''

def calculate_real_estate_investment(savings, home_price = 100000, rental_income = 1000,
    annual_appreciation = 0.03, mortgage_rate = 0.05,
    loan_term_years = 30, months = 12): # defaulting

    down_payment = months * savings
    loan_amount = home_price - down_payment
    rate = mortgage_rate / 12
    term_total_months = loan_term_years * 12
    if down_payment < home_price:
        monthly_mortgage_payment = (loan_amount * rate * ((1 + rate) ** term_total_months)) /
        (((1 + rate) ** term_total_months) - 1) # long equation
    else:
        monthly_mortgage_payment = 0
        loan_amount = 0

    total_mortgage_paid = monthly_mortgage_payment * months
    remaining_loan_balance = loan_amount - total_mortgage_paid

    total_rental_income = rental_income * months

    appreciated_value = home_price * (1 + (annual_appreciation / 12)) ** months

    total_investment_value = appreciated_value + total_rental_income - remaining_loan_balance
    total_investment_value = round(total_investment_value, 2)

    return total_investment_value

'''

the main function puts all the previus functions together to make one working program. It
formats all previous programs nicely to make it readable for
the user.
Parameters:
    none
Return:
    none
'''

def main():
    print('Welcome to the 2025 Money Decision Guide!')
    print()
    spare_cash = calculate_spare_cash()
    if spare_cash <= 0:
        print('\nYou are spending more than or equal to what you earn. Try reducing

```

```

        expenses!')
else:
    print('\nYou have $' + str('{:.2f}'.format(spare_cash)) + ' available to save or
        invest each month.')
    print()
    print('***Projected Savings After One Year**')
    print()
    print('Regular Bank Account: $' + str(calculate_savings(spare_cash, 0.0001, 12)))
    print('High-Yield Savings Account (HYSA): $' + str(calculate_savings(spare_cash,
        0.045, 12)))
    print('S&P 500 Investment: $' + str(calculate_sp500_investment(0.08, spare_cash,
        0.015, 12)) + ' (estimation, market varies)')
    print('Real Estate Investment Value: $' +
        str(calculate_real_estate_investment(spare_cash, 100000, 1000, 0.03, 0.05, 30,
        12))
        + ' (includes rental income & appreciation)')
    print()
    print("Let's switch it up a notch, what happens if this trend continues for 3 years")
    print()
    print('***Projected Savings After Three Year**')
    print('Regular Bank Account: $' + str(calculate_savings(spare_cash, 0.0001, 36)))
    print('High-Yield Savings Account (HYSA): $' + str(calculate_savings(spare_cash,
        0.045, 36)))
    print('S&P 500 Investment: $' + str(calculate_sp500_investment(0.08, spare_cash,
        0.015, 36)) + ' (estimation, market varies)')
    print('Real Estate Investment Value: $' +
        str(calculate_real_estate_investment(spare_cash, 100000, 1000, 0.03, 0.05, 30,
        36))
        + ' (includes rental income & appreciation)')

'Calling the function'

if __name__ == '__main__':
    main()

```